

Projektowanie aplikacji mobilnych

Ćwiczenia #6 – Testy jednostkowe

Grupa:

Ilias Abdeljalil (40028)

1. Napisać testy jednostkowe dla klasy BookStore (3pkt)

Kod klasy BookStore należy skopiować z przykładowego projektu:

Android: https://github.com/kmate-dev/PAM_Android/blob/main/app/src/main/java/com/kmate/dev/pam_android/fortesting/BookStore.kt

iOS: https://github.com/kmate-dev/PAM_iOS/blob/main/PAM_iOS/Fortesting/BookStore.swift

Wymagane testy:

- 1 dowolny test metody calculateTotalValue
- 1 dowolny test metody getBooksByAuthor
- Test parametryczny metody isAuthorInCollection
- Kilka testów metody addBook (należy sprawdzić, czy książka jest dodawana poprawnie, ale również czy wyjątek jest rzucony w odpowiednich przypadkach)

Uwaga! Poprawnie napisany test parametryczny powinien wywołać się jako kilka testów po uruchomieniu. Inaczej mówiąc... nie chodzi o to, żeby przetestować kilka scenariuszy w ramach jednego testu.

Screenshoty napisanych testów oraz ich wyników:

```

package com.kmate.dev.pam_android.fortesting

import org.junit.jupiter.api.Assertions.*
import org.junit.jupiter.api.Test
import org.junit.jupiter.api.assertThrows
import io.kotest.core.spec.style.StringSpec
import io.kotest.matchers.shouldBe

class BookStoreTest {

    private val sampleBooks = listOf(
        Book(1, "Book One", "Author One", 10.0),
        Book(2, "Book Two", "Author Two", 20.0),
        Book(3, "Book Three", "Author One", 30.0)
    )

    @Test
    fun `calculateTotalValue should return the correct sum of book prices`() {
        val bookStore = BookStore(sampleBooks)
        val totalValue = bookStore.calculateTotalValue()
        assertEquals(60.0, totalValue, "The total value of books should be 60.0")
    }

    @Test
    fun `getBooksByAuthor should return books by the specified author`() {
        val bookStore = BookStore(sampleBooks)
        val booksByAuthorOne = bookStore.getBooksByAuthor("Author One")
        assertEquals(2, booksByAuthorOne.size, "Author One should have 2 books")
        assertTrue(booksByAuthorOne.all { it.author == "Author One" }, "All books should be authored by Author One")
    }

    @io.kotest.core.spec.style.StringSpec
    class ParameterizedIsAuthorInCollectionTest : StringSpec({
        val sampleBooks = listOf(
            Book(1, "Book One", "Author One", 10.0),
            Book(2, "Book Two", "Author Two", 20.0)
        )
        val bookStore = BookStore(sampleBooks)

        "isAuthorInCollection should return true if author exists in the collection" {
            bookStore.isAuthorInCollection("Author One") shouldBe true
        }

        "isAuthorInCollection should return false if author does not exist in the collection" {
            bookStore.isAuthorInCollection("Unknown Author") shouldBe false
        }

        "isAuthorInCollection should ignore case when checking author existence" {
            bookStore.isAuthorInCollection("author one") shouldBe true
        }
    })

    @Test
    fun `addBook should add a book to the collection`() {
        val bookStore = BookStore(mutableListOf())
        val newBook = Book(4, "Book Four", "Author Four", 40.0)

        bookStore.addBook(newBook)

        val allBooks = bookStore.getAllBooks()
        assertTrue(allBooks.contains(newBook), "New book should be added to the collection")
    }

    @Test
    fun `addBook should throw an exception if a book with the same ID already exists`() {
        val bookStore = BookStore(sampleBooks)
        val duplicateBook = Book(1, "Duplicate Book", "Author Duplicate", 50.0)

        val exception = assertThrows<IllegalArgumentException> {
            bookStore.addBook(duplicateBook)
        }

        assertEquals("A book with the same ID already exists.", exception.message)
    }

    @Test
    fun `addBook should add multiple unique books`() {
        val bookStore = BookStore(mutableListOf())
        val book1 = Book(5, "Book Five", "Author Five", 50.0)
        val book2 = Book(6, "Book Six", "Author Six", 60.0)

        bookStore.addBook(book1)
        bookStore.addBook(book2)

        val allBooks = bookStore.getAllBooks()
        assertTrue(allBooks.containsAll(listOf(book1, book2)), "Both books should be added to the collection")
    }
}

```

2. Napisać test parametryczny dla metody calculateTotalValue (1pkt)

Do napisania tego testu potrzebna będzie klasa pomocnicza, która będzie generowała pary: listę książek oraz ich sumaryczną cenę. Cena książek powinna być inna przy każdym wywołaniu testu.

W przypadku Androida proszę użyć `@ArgumentsSource` oraz `ArgumentsProvider`.

W przypadku iOS będzie to po prostu funkcja zwracająca taką listę par danych.

Screenshotty testu oraz klas pomocniczych:

```
package com.kmate.dev.pam_android.fortesting

import org.junit.jupiter.params.provider.Arguments
import org.junit.jupiter.api.extension.ExtensionContext
import org.junit.jupiter.params.provider.ArgumentsProvider
import java.util.stream.Stream
import kotlin.random.Random

class RandomBooksProvider : ArgumentsProvider {
    override fun provideArguments(context: ExtensionContext?): Stream<out Arguments> {
        val random = Random(System.currentTimeMillis())
        return Stream.generate {
            val numberOfBooks = random.nextInt(1, 10) // Generuje liczbę książek od 1 do 10
            val books = (1..numberOfBooks).map {
                Book(
                    id = it,
                    title = "Book $it",
                    author = "Author ${it % 3}", // Autorzy powtarzają się cyklicznie
                    price = random.nextDouble(5.0, 50.0) // Generuje cenę od 5.0 do 50.0
                )
            }
            val totalPrice = books.sumOf { it.price }
            Arguments.of(books, totalPrice)
        }.limit(10) // Generuje maksymalnie 10 zestawów danych
    }
}
```

```

package com.kmate.dev.pam_android.fortesting

import org.junit.jupiter.api.Assertions.assertEquals
import org.junit.jupiter.params.ParameterizedTest
import org.junit.jupiter.params.provider.ArgumentsSource

class BookStoreParameterizedTest {

    @ParameterizedTest
    @ArgumentsSource(RandomBooksProvider::class)
    fun `calculateTotalValue should return correct sum of prices for random books`(
        books: List<Book>,
        expectedTotalPrice: Double
    ) {
        val bookStore = BookStore(books)
        val actualTotalPrice = bookStore.calculateTotalValue()
        assertEquals(expectedTotalPrice, actualTotalPrice, 0.001, "Total price does not match")
    }
}

```

3. Zagadka: Jak naprawić klasę BrokenBookStore tak, żeby stała się testowalna? (2pkt)

Kod klasy BrokenBookStore należy skopiować z przykładowego projektu:

Android: https://github.com/kmate-dev/PAM_Android/blob/main/app/src/main/java/com/kmate/dev/pam_android/fortesting/BrokenBookStore.kt

iOS: https://github.com/kmate-dev/PAM_iOS/blob/main/PAM_iOS/Fortesting/BrokenBookStore.swift

Będzie potrzebna także klasa LocalBookStoreImpl, również należy ją skopiować z przykładowego projektu:

Android: https://github.com/kmate-dev/PAM_Android/blob/main/app/src/main/java/com/kmate/dev/pam_android/fortesting/LocalBookStoreImpl.kt

IOS: https://github.com/kmate-dev/PAM_iOS/blob/main/PAM_iOS/Fortesting/LocalBookStoreImpl.swift

Klasa BrokenBookStore została napisana w taki sposób, że jej testowanie jest niemożliwe. Celem zadania jest dokonanie takich modyfikacji w klasach

BrokenBookStore oraz LocalBookStoreImpl, żeby możliwe było napisanie jakiegokolwiek testu jednostkowego.

Screeny naprawionych klas:

Screen przykładowego testu klasy BrokenBookStore: